JAVA OOP LABORATORY EXERCISE

Campus Café Ordering System

Name: John Michael Bacang

Section: _____

Date: November 7, 2026

Project Description

This project is a Java console application that processes customer orders in a campus café. A customer may order up to three menu items. The system supports beverages, pastries, and meals, calculates their prices, applies a student discount or voucher, and awards loyalty points.

OOP Concepts Used

Classes and Objects: The program uses seven classes: MenuItem, Beverage, Pastry, Meal, Customer, Order, and CafeApp. CafeApp creates customer, menu item, and order objects.

Abstraction: MenuItem is an abstract parent class. Its abstract computePrice() method requires each menu item subclass to provide its own pricing calculation.

Encapsulation: Instance variables are private or protected. Public getters and setters control access, and setters reject invalid values such as a negative price or negative number of extra sides.

Constructors and Overloading: Default and parameterized constructors initialize menu items. Customer has three overloaded constructors. The program also overloads addPoints() and computeTotal().

Inheritance and Overriding: Beverage, Pastry, and Meal extend MenuItem. Each subclass overrides computePrice() and getDescription() to provide behavior suited to its item type.

Polymorphism: Order stores menu items using MenuItem references. Calling computePrice() on each item runs the correct subclass implementation.

Conditional Statements: if and if-else statements validate input, calculate size and warming fees, check the three-item order limit, and apply the student discount.

Object Interaction: Order stores a Customer and up to three MenuItem objects. Printing a receipt calculates the total and updates the customer's loyalty points.

Program Output

The Java files were compiled and run successfully. The following is the actual console output:

```
Order is full. Cannot add Cookie [P-02].

Order #1001 - Ana Reyes (Student)
Iced Latte [B-01] Size: L ............ 120.00
Ensaymada [P-01] (Warmed) ............ 55.00
Chicken Adobo [M-01] Upsized, 1 side ............ 142.00
Subtotal: 317.00
Student discount (10%): -31.70
TOTAL: 285.30
Loyalty points: 5
Error: base price cannot be negative.

Order #1002 - Ben Cruz
Hot Chocolate [B-02] Size: M ............ 95.00
Beef Tapa [M-02] Regular, 0 sides ............ 130.00
Subtotal: 225.00
Voucher: -20.00
TOTAL: 205.00
Loyalty points: 4

Default objects created: Unnamed Item [N/A] Size: S, Unnamed Item [N/A], Unnamed Item [N/A] Regular, 0 sides
```

Challenges Encountered and Solutions

One challenge was applying different price rules to each menu item while keeping the order class simple. This was handled through an abstract MenuItem class and overridden computePrice() methods. Another challenge was preventing invalid data. Validation was added to setters, including the base price and extra sides. The order limit was handled by storing three separate MenuItem fields and checking each empty slot before accepting another item. The program was tested with javac and ran without compilation errors.

Submission Note

The project ZIP contains the seven Java source files and OUTPUT.txt. Upload the project folder to GitHub and submit the repository link as required by the activity. Replace the blank section field on the cover page before submitting.